AMERICAN LATIN CLASS ATTENDANCE SYSTEM

Technology Stack

Table of Contents

Technology Stack Decision

Backend Framework

Selected framework: **Express.js on Node.js**.

Reason:

- Fits the existing backend-first academic project.
- Supports clean REST APIs, middleware, validation, controllers, services and tests.
- Keeps the codebase understandable for Advanced Web Development review.

Frontend Framework

Selected and implemented framework: **React + Vite**.

Reason:

- React is widely accepted for academic and professional web applications.
- Vite gives a simple, fast frontend build pipeline.
- The landing page and private dashboard are implemented as React components under `06Code/frontend`.

Implemented frontend components:

- `LandingPage`
- `LoginPage`
- `EnrollmentForm`
- `PrivateDashboard`
- `AttendanceWorkflow`
- `TeacherCheckInWorkflow`
- `ReportsPanel`
- API client module using `fetch` with credentials.
- Google Identity Services integration through the React login page.

ORM

Selected ORM: **Prisma ORM**.

Reason:

- Strong PostgreSQL support.
- Schema is explicit and easy to review academically.
- Generated client reduces raw SQL/data mapping mistakes.
- Works well with service/repository architecture.

Current implementation status:

- `06Code/prisma/schema.prisma` defines the normalized PostgreSQL data model.
- `PrismaDatabaseContext` and `PrismaRepository` wire Prisma into the repository layer.
- `DB_DRIVER=prisma` is now the production/default database driver when `DATABASE_URL` exists.
- In-memory repositories remain for fast unit/integration tests.

Legacy note:

- The previous raw `pg` repository remains as fallback with `DB_DRIVER=pg`, but the required production ORM path is Prisma.

Python API Framework

Selected framework: **FastAPI**.

Reason:

- Satisfies the additional Python API requirement without weakening the existing Node.js backend architecture.
- Provides automatic OpenAPI metadata, clean route definitions and strong support for JSON APIs.

- Works well as a small analytics microservice behind Nginx or an AWS load balancer.

Current implementation status:

- The Python API lives under `06Code/python-analytics-api`.
- It exposes `/api/analytics/v1` endpoints for attendance risk, scholarship readiness, branch performance and teacher workload.
- It reuses the JWT session token issued by the Node Auth API and validates the token against the shared PostgreSQL `sessions` table.
- It is designed for a separate AWS EC2 instance on port `8000`.

Implemented React + Vite Frontend

The frontend migration is now implemented under `06Code/frontend` with React components for the public landing page and private dashboard workflows. Vite builds the browser application into `06Code/dist/frontend` using `npm run frontend:build`.

Implemented components:

- `LandingPage`, `EnrollmentForm`, `BranchSummary`, and `StylesLevels` for the public enrollment experience.
- `LoginPage` for Google Sign-In.
- `PrivateDashboard`, `AttendanceWorkflow`, `TeacherCheckInWorkflow`, `ReportsPanel`, `AuthStatus`, and `LogoutButton` for authenticated operations.

Deployment note:

- Express serves `06Code/dist/frontend` first. The private dashboard route is handled by the React app, and legacy `/public/private` files are not used.
- The hero asset is available through the Vite public assets folder and is included in the frontend build.
- AWS EC2 deployments should run `npm run frontend:build` before `npm start`; Nginx may also serve `06Code/dist/frontend` directly while proxying `/api/v1` to Express.